

Role models — patterns of agent system analysis and design

E A Kendall

Due to their autonomy and social behaviour, agents will play important roles in emerging enterprises; they will fill key positions and provide essential capabilities. This paper describes role modelling as a software engineering technique for specifying, analysing, and designing systems on the basis of the roles that the agents will play. This paper builds on our earlier research in patterns of agent systems. A pattern is a useful solution to a reoccurring problem in a particular context; patterns can be used to solve software engineering problems in analysis, design, and implementation. This paper explains how object-oriented role models can be extended to represent patterns of agent interaction that can then be employed to engineer agent systems. Role theory deals with collaboration and co-ordination. Roles have also been applied to distributed systems management, and to agent and robot systems. However, there has to date not been a methodology for realising these representations in an automated or semi-automated system, due to the lack of adequate formalism and corresponding abstractions in software. Role models rectify this situation for software analysis, design, and implementation.

1. Introduction

Agents are autonomous, reactive, proactive, and social. Their reactivity allows them to adjust to changes, while their proactivity allows them to manifest desired changes or effects via goal-driven behaviour. Networked, distributed enterprises are difficult to manage; it is not likely that people can directly manage or manipulate every facet. However, agent autonomy means that agents can work on their own, without human intervention, if necessary. Based on these properties, agents are vital to the information technology infrastructure of tomorrow's emerging and networked enterprises.

Interaction and collaboration are essential aspects of any agent system. However, software engineering methodologies have not to date adequately emphasised collaborations. Recent developments in role modelling provide new abstractions and techniques for specifying and designing systems on the basis of patterns of interactions. This paper presents an approach to agent system engineering based on extensions to object-oriented role modelling. The approach is pattern-based; the aim here is to develop a pattern language or living, modular methodology for agent engineering. As discussed in section 3, the basis of our technique for introducing agent technology into mainstream software engineering is a catalogue of agent role models, complete with specifications, designs, and sample implementations.

Other agent researchers have employed the concept of an agent role [1—5]. However, this paper presents a methodology for identifying, specifying, designing, and implementing agent systems on the basis of agent roles and role models. Further, this research also aims to:

- define the concepts of roles and role models with respect to agent systems,
- clarify, compare, and contrast work on roles and role models from object-oriented software engineering, business process management, and agent systems.

Section 2 provides background information on role models in object-oriented software engineering, while section 3 defines agent roles as extensions of those played by objects. Sections 4 and 5 utilise examples to explain how role modelling can be used to facilitate agent system analysis and design, respectively.

Section 6 compares role modelling to other techniques for agent system engineering; it also provides a summary and conclusions.

2. Role models in object-oriented software engineering

2.1 Overview

In object-oriented software engineering, roles and role models [6—15] have been developed in response to a recognition that there are important facets of systems that are difficult to model using only classes. Whereas classes stipulate the capabilities of individual objects, role models emphasise how objects interact with each other. A role model describes a structure of interacting objects. The description is comprised of roles and the relationships between them. A role defines a position and a set of responsibilities within a role model. The responsibilities in a role are made up of services and tasks. A role has collaborators; these are other roles with which it interacts.

Consider an analogy from human organisations. An organisation such as the civil service will contain many committees. While each committee differs in detail, there is a common pattern of roles and relationships. One can thus define a committee role model, containing the roles of chairperson, secretary, treasurer, etc. Officers appointed to roles in a new committee acquire responsibilities appropriate to that role. Experienced officers will know their responsibilities and their relationships with other committee members because they understand how committees run. It is this knowledge of how committees run that constitutes the committee role model.

It should be noted that:

- role models are orthogonal and complementary to class models — they do not replace them,
- like classes, role models are first class objects (they can be instantiated, generalised, specialised, and aggregated into compound models [9]), promoting reuse — a class can be used to instantiate individual objects, while a role model can be used to instantiate a sub-system of interacting objects (as illustrated by the example in section 2.2, role models are similar to UML collaboration diagrams [16] — however, role models are more reusable and extensible); once you have a base role model, you can build on it to yield other, more advanced ones,
- role models can be dynamic [9], involving sequencing, evolution, and role transfer — this could be of value to models of mobility and adaptability.

There are three ways that role models can be synthesised from existing ones:

- a new role or role model may be derived from one or more base models — in this case, the derived role must be able to play the base roles; usually there is more than one base model, similar to multiple inheritance (multiple inheritance is often employed during analysis, but not carried forward into design and implementation);

- one role or role model may be an aggregate of other roles or role models — this may be encapsulated, visible or virtual:
 - if it is visible, this means that a single role in the outer system role model is able to visibly play component roles, i.e. interface will reflect the embedded, visible roles,
 - if it is encapsulated or hidden, the component roles are only found in the internal view of the role, i.e. the role itself may use the component roles, but no external roles can do this,
 - virtual aggregation is only conceptual, i.e. a role model can be represented, with multiple roles in it, by a single virtual role;
- roles in a role model can also constrain each other — two roles may imply each other, meaning that a single entity must play both roles; synergy [13—15] can make a combined role more than just the sum of the parts.

2.2 Illustration from object-oriented software engineering

The following role model example (based on Reenskang [12]) is a high-level view of process management in an enterprise. There are customers and managers, including the plant manager. There are several functional groups — assembly, QA (quality assurance), and a repair shop.

The example is first presented using UML (unified modelling language). This is the pre-eminent modelling language for object-oriented analysis and design. It features four perspectives — class, data, interface, and collaboration. Figure 1 provides the class diagram, while an instance collaboration diagram appears in Fig 2. In the class diagram, as a triangle indicates inheritance, the three functional groups shown are subclasses of the class Functional Group. Further, Plant Manager is a subclass of Manager. The solid lines indicate a static relationship, which shows that instances of the Functional Group subclasses have a static relationship with an instance of the Plant Manager class. There are also static relationships between Customer and Manager, and between the Manager and Functional Group classes.

While rich in information content, the class diagram does not depict the interactions between objects — this is addressed in Fig 2. Individual objects (instances of the classes in Fig 1) appear, and they collaborate with each other through messaging. A typical messaging sequence is shown. The sequence begins with aCustomer's (an instance of class Customer) request for a new product (1). This leads to messages 2 through 5, where aPlant Manager delegates work to anAssembly and then to aQA. The product is then delivered to the customer in message 6. All messaging is done through aPlant Manager.

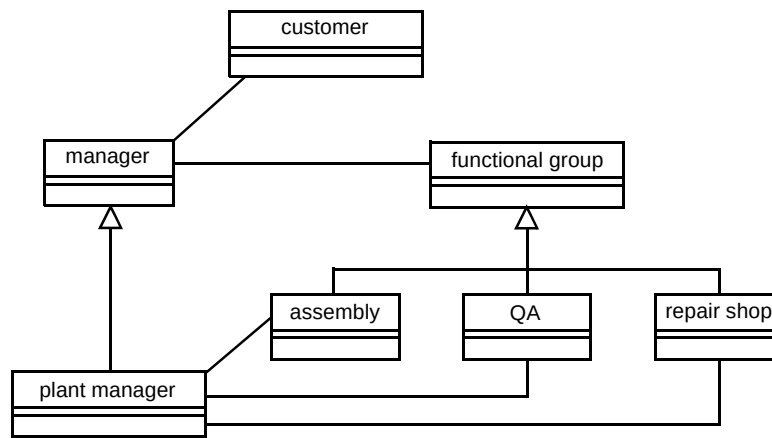


Fig 1 Class diagram in UML notation.

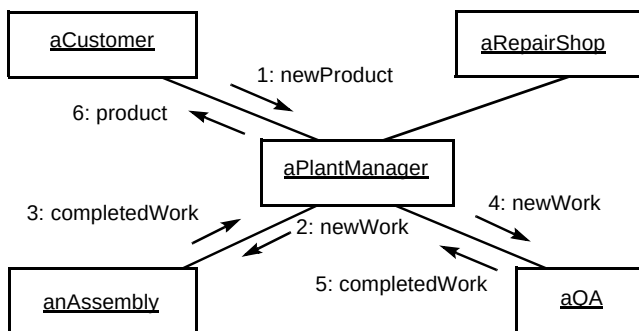


Fig 2 Instance collaboration diagram in UML notation.

The collaboration depicted in Fig 2 is a pattern of general applicability; it is characteristic of centralised control. In Gamma et al [17], it is called the Mediator pattern. However, the instance collaboration diagram (Fig 2) does not capture the interaction as a pattern; it only shows it as a collaboration between instances of classes in a particular application. A role model representation of the pattern of interaction is shown in Fig 3, using a combination of the role model notation found in Anderson [6], Reenskaug et al [10], Riehle [12—14]. With this notation, a

rounded box is a role, and the solid arrows indicate collaboration paths between the roles. The direction of the arrow represents the direction of messaging, and the solid circle on the link from the Mediator to the Colleague indicates that there is more than one Colleague. As can be seen from Fig 3, the Colleague roles do not interact directly with each other. All collaboration occurs through the Mediator role.

Once a role model has been captured, it can be instantiated, as needed, in any application. The five object instances in the enterprise application (shown as rectangles in the bottom half of Fig 3) can be assigned to play the various roles, as indicated by the dashed arrows. The dashed arrows signify that aCustomer plays the role of Client, while aPlantManager is assigned the role of Mediator. The other three objects (aQA, anAssembly, and aRepairShop) all play the role of Colleague.

The full semantics of the role model for the Mediator pattern can be detailed in additional views and notation as described elsewhere [6, 10, 12—14]. The important distinction between the collaboration diagram (Fig 2) and

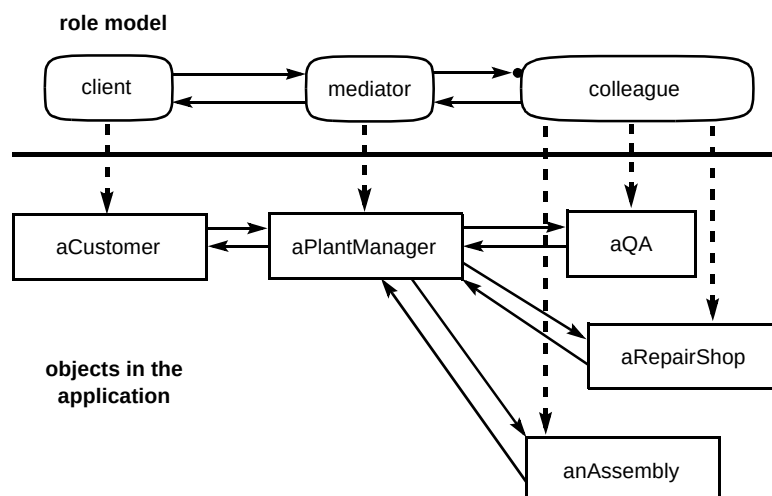


Fig 3 Role model for the mediator pattern and role assignments.

the role model (Fig 3) is that the role model is an abstraction; the instance collaboration diagram is merely an instantiation of it.

Another form of a collaboration diagram (called a class collaboration diagram) was introduced in UML Version 1.1 [16]. This diagram can express what is shown in Fig 3. The UML Version 1.1 collaboration diagram notation for the Mediator example is provided in Fig 4. The Mediator collaboration is shown as a dashed ellipse. There are three roles indicated by labels on the dashed lines — Client, Mediator and Colleague. The five objects play the five roles, with aQA, anAssembly, and aRepairShop all playing the role of colleagues. The UML version 1.1 class collaboration diagram (Fig 4) is equivalent to Fig 3. However, the collaboration cannot be separated from the entities that are instantiating it. Additionally, the collaboration diagram (Fig 4) cannot be generalised, abstracted, or aggregated (section 2.1). Unlike role models, collaboration diagrams are not first class objects.

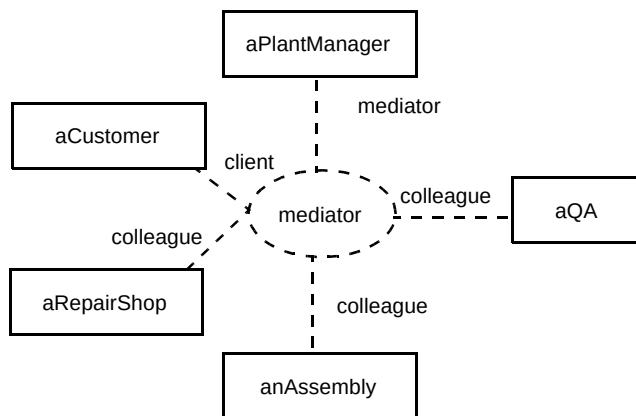


Fig 4 UML Version 1.1 collaboration diagram for Mediator example.

2.3 Role synergy — the bureaucracy pattern

In role synergy, several roles are merged into a new composite role that is more than a sum of the individual roles. An example of this is the Bureaucracy pattern [14]. This pattern is similar to actual bureaucracies, where there is a long chain of responsibility, a multilevel hierarchical organisation, and centralised control. This pattern can be constructed in software by bringing together the Composite, Mediator, Observer, and Chain of Responsibility patterns [17].

The Bureaucracy pattern is formed from multiple Mediators and Colleagues, arranged in a Chain of Responsibility. Because a Mediator may in fact represent several Mediators, the Composite pattern also appears. A Mediator is also an Observer of the colleagues under its control. A simple combination of the four design patterns results in sixteen roles for the Bureaucracy pattern, as follows:

- Handler Client, Handler, Successor, Predecessor, Tail, Tail Client — Chain of Responsibility pattern,
- Mediator, Colleague — Mediator pattern,
- Observer, Subject — Observer pattern,
- NodeClient, Node, Parent, Child, Root, Root Client — Composite pattern.

However, role synergy occurs in the Bureaucracy pattern; the resulting compound pattern is more than a sum of the four individual ones. Many of the roles can only be combined in a certain way (these are roles that constrain or limit one another); additionally, some roles are merged. Role synergy yields that only six roles appear in the bureaucracy pattern.

The six roles in the Bureaucracy pattern are the Director, Director Client, Manager, Subordinate, Clerk, and Clerk Client. The Director is at highest level of the Bureaucracy. Managers appear at the next level, and multilevel management structures are possible. Clerks are at the lowest level in the Bureaucracy; they provide services for a ClerkClient. Role synergy occurs because, in the Bureaucracy pattern, the same object (a Manager) must play the roles of Mediator, Parent, Observer, Predecessor, Successor and Child. Additionally, an object that plays the role of Subordinate must be a Colleague, a Subject, a Successor, and a Child. In addition to playing many roles, the objects have to be able to switch contexts.

Figure 5 depicts the role model for the Bureaucracy pattern. Managers and Subordinates must also be Clerks; this is indicated by the triangle for role refinement. A Director refines or adds capabilities to a Manager, and this additional behaviour is for error handling and managing the entire hierarchy.

3. Role models of agent systems

3.1 Overview

Section 2 illustrated how role modelling can be employed during object-oriented analysis. Over the last few years, research into how object-oriented techniques can be utilised and extended for agents systems [18—21] has been completed. Role modelling offers a promising approach for agent analysis and design; many of the advantages that role models bring to object-oriented software apply also to agents. Additional advantages apply, because social behaviour is an important aspect of multi-agent systems. Role models provide an abstraction that can unify diverse aspects of an agent system — acquaintance models, collaboration protocols, task models, etc. Furthermore, agents, objects, and people can all play roles, so that a role model can span multiple layers in a software system.

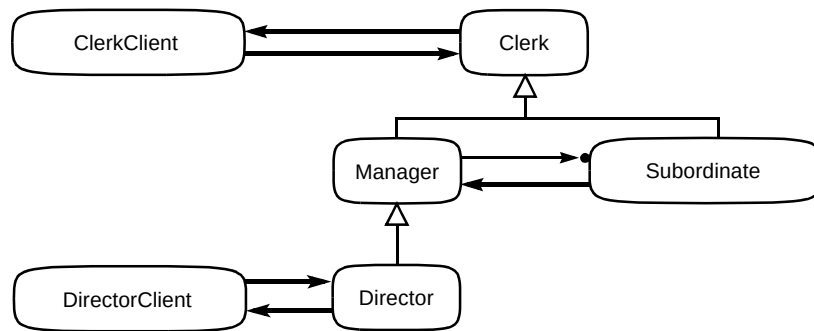


Fig 5 Role diagram for the Bureaucracy pattern.

Adopting role modelling in the design of agent systems means that advantage can be taken of existing work in object-oriented design, for example:

- Barbuceanu [1] partitions an agent's beliefs, goals, and behaviour according to the roles that it plays, in order to facilitate context switching and to decompose sophisticated agent behaviour into simpler modules — roles have also been used to partition an object's behaviour [22],
- role model synergy or synthesis, which involves integrating roles, is valuable for agent design,
- role models have been demonstrated to be valuable for framework, subsystem, and system design [6, 13, 15],
- there are several existing object-oriented design patterns for roles [22, 23] that are potentially useful for agents,
- role models are not static — they can have various forms of dynamic behaviour, for example, agent organisations can take on various formations (role models), and role assignments can be assigned dynamically [5, 24], role model dynamics may be valuable for modelling mobility, adaptive behaviour, context switching, and other aspects of agent systems.

This section shows how the concept of an object role and some of the role models described above can be extended and applied to the agent domain.

3.2 Facets of agent roles

Based on section 2, object roles have the facets or dimensions as shown in Table 1.

As agents are extensions of objects [18—21], they encompass all the features that objects have, adding autonomous, proactive, social, reactive, and intelligent behaviour. Therefore, additional facets are required, as shown in Table 2 [25]. Table 2 reflects how agents encompass behaviour from objects, organisational theory, and artificial intelligence. An agent role has more responsibilities than an object role — this is due to its goals,

obligations, and interdictions. An agent role also encompasses expertise and behaviour for co-ordination and negotiation. (Some of the facets listed in Table 2 are preliminary, e.g. resources may not belong to an agent role.)

Table 1 Facets of an object role.

role model	context
responsibilities	services, tasks
collaborators	roles with which it interacts
external interfaces	access to services
relationships to other roles	aggregation, generalisation, refinement, role sequences

Table 2 Facets of an agent role.

role model	context
responsibilities	services, tasks, goals, obligations, interdictions
collaborators	roles with which it interacts
external interfaces	access to services
relationships to other roles	aggregation, specialisation, generalisation, role sequences
expertise	ontology, inferencing, problem-solving knowledge
co-ordination and negotiation	protocol, conflict resolution, knowledge of why other roles are related, permissions
other	resources, learning/adaptability

3.3 Supply chain with service provisioning

A supply chain is a common pattern of collaboration for agents [1, 26]. A supply chain is comprised of suppliers and consumers. The head of the supply chain is purely a consumer, while the tail is purely a supplier. An internal link in the chain must be both a consumer and a supplier. A consumer can have many suppliers, but a supplier usually only has one consumer in a given supply chain. (The supplier will typically be involved in several supply chains, however.)

The supply chain pattern of interaction has been represented in terms of role models. At the highest level, a supply chain is made up of three roles — Supply Chain (SC) Head, SC Tail, and SC Participants. The relationships between the links in the chain can be captured in terms of another role model — Predecessor-Successor. A Predecessor can have many Successors, but a Successor has only one Predecessor. As shown in the role model in Fig 6, an SC participant is both a Predecessor and a Successor. The SC Participant role merges the roles of Predecessor and Successor, refining the behaviour of both. An SC Head is a specialisation of a Predecessor, and an SC Tail refines the role of successor.

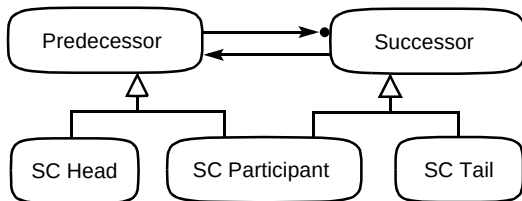


Fig 6 Supply Chain — top level role model.

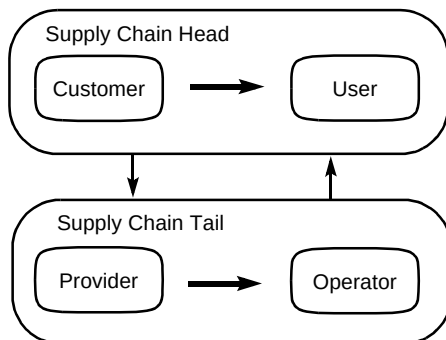


Fig 7 Supply Chain — role sequences.

Many detailed supply chain role models that consider service provisioning and the phases of negotiation, delivery, and payment have been documented. Figure 2 considers negotiation and delivery in a supply chain with just two elements. Because of the two phases, a role sequence is required, and nested or contained roles are shown. A filled chevron signifies the order of the sequence. In Fig 7, the SC Head goes from being a Customer who negotiates for services to a User who utilises them. An SC Tail is first a provider and then an operator. (The names for the Customer, User, Provider, and Operator roles follow BT terminology for service provisioning.) During negotiation, the Customer in the head negotiates with the Provider in the tail. During delivery, the Operator in the successor delivers supplies to the User in the predecessor.

When a supply chain consists of more than two elements, each internal link must be able to support all four internal roles — Customer, Provider, User, and Operator. During negotiation, the pattern commences at the SC Head, and finishes at the SC Tail. During delivery, the opposite is true. When the entire chain is considered, it can be seen that

an SC Participant has a different role sequence from those in Fig 7. Its role sequence is as follows: i) Provider, ii) Customer, iii) Provider, iv) User, v) Operator.

Additional notations and formalisms for role models can be found in Andersen [6]; here role responsibility cards (RRCs) [18] have been introduced as a simple way to document the key facets of an agent role in a given role model (Table 2). Sample, condensed RRCs for the SC Predecessor, SC Successor, and SC Participant roles are provided in Fig 8. The responsibilities on the cards are actually further delegated to the detailed roles that appear in the role sequences, and some of the role facets have been omitted.

Role: Supply Chain (SC) Head	
Responsibilities:	Collaborators:
initiative and complete supply negotiation	<> (send, receive message) Successors
receive supplies	< (receive message from) Successors

Role: Supply Chain (SC) Tail	
Responsibilities:	Collaborators:
complete supply negotiation	<> (receive, send message) Predecessor
produce supplies	
deliver supplies	> (send message to) Predecessor

Role: Supply Chain (SC) Head	
Responsibilities:	Collaborators:
complete supply negotiation	<> (receive, send message) Predecessor
initiate and complete supply negotiation	<>(send, receive message) Successors
receive supplies	< (receive message from) Successors
produce supplies	
deliver supplies	> (send message to) Predecessor

Fig 8 Role responsibility cards for Supply Chain role model.

4. Role models during agent system analysis

4.1 Patterns

Another concept that is proving valuable in object-oriented software engineering and which can be fruitfully applied to agents is that of patterns [17, 19, 20, 22]. Alexander initially developed the concept of patterns in his work on architecture and urban planning. Patterns are masterful solutions that take time to evolve or develop; they can be identified once the level of understanding is deep and comprehensive enough to uncover invariants. Individual patterns are not isolated; each pattern depends on other patterns. There is a network of connections between patterns; these comprise the pattern language. A pattern

language is a set of symbols and elements, and rules for combining them. A pattern language is not static, it evolves, as needed. Patterns that are no longer used die off, while new patterns appear and are added. This is what makes a pattern language a living and modular methodology that can adapt to change. Patterns are neither abstract nor vague; they lead to concrete systems. A pattern language is a language of experience. When an experienced analyst builds a model or when an experienced designer performs a design, they do not start from scratch; they combine patterns in their mind.

4.2 Agent role models as patterns

Agent analysis should begin with goals [27] and goal centric use cases [21]. Goals should be partitioned, which means that they should be assigned to individual roles. Any agent application will in fact encompass many role models. During analysis, relevant role models should be identified. This can be done from scratch by identifying the agents in an application and their collaborations. Alternatively, a role model catalogue can be used; this provides commonly occurring agent role models, which means that previous analysis experience can be reused. A role model catalogue resembles a set of patterns or a pattern language, and role model catalogues are being used at BT for downstreaming agent technology. Whereas agent technology is still relatively new, some simple patterns have emerged for agent interaction. These patterns are documented in the role model catalogue, and they should be utilised to disseminate expertise in agent technology.

The role model catalogue developed at BT includes approximately 60 role models, including those from agent-enhanced workflow, flexible manufacturing, electronic commerce, agent-based information management and retrieval, and the FIPA protocols (contract net, auctions, etc). As discussed in sections 2 and 3, these role models are patterns of interaction. The patterns in the BT catalogue were mined from numerous existing agent frameworks or specifications. This includes Zeus [26], FIPA [28], and Single Function Agents [29]. The roles and role models found in intelligent systems, such as those found in KADS [30], have also been considered.

The BT catalogue provides concrete details about the roles and the role models. All the facets from Table 2 are considered, in addition to interactions and protocols. For example, the supply chain roles can employ certain techniques for negotiation and co-ordination, such as a contract net or an auction; further, some of the role models involve ontologies.

When presented with a new application, analysts should go through the role model catalogue to identify which patterns of interaction appear. To facilitate dissemination and proper use, full pattern format has been used for the role

model documentation, i.e. the role models are fully motivated solutions for application-analysis models.

Kristensen [9] discusses roleification, or the process of associating a role with a concept. This involves classification, where you organise or arrange how an object is viewed by other objects according to categories of roles. This means that you need to have categories of roles that an object can play. Once you have categories of roles and role models, they become role types and role model types that can then be exemplified or instantiated in a particular role model. Roleification also involves some kind of restriction or narrowing of a concept. You take a perspective or view of it within a certain context. The BT catalogue of agent roles is based on Kristensen's ideas on roleification, and the agent roles have been organised into various categories.

4.3 Illustration from an agent application

As an illustration, consider an application in agent enhanced workflow (AEW) [26] or flexible manufacturing. The agents can be arranged in such a way that each agent is dependent upon one other agent to deliver work or supplies to them. For example, four agents may represent an end customer and three enterprises, respectively. The Customer deals directly only with Enterprise 1. Enterprise 1 depends on Enterprise 2 for supplies or work, and Enterprise 2 in turn depends on Enterprise 3. At the highest level, the application is an instantiation of the Supply Chain role model. The Customer is the SC Head, Enterprise 1 and Enterprise 2 are both SC Participants, and Enterprise 3 is an SC Tail. Enterprise 1 is an SC Successor to the Customer, but it is an SC Predecessor to Enterprise 2.

However, each enterprise in the supply chain can be made up of several entities. For example, Enterprise 1 may be a manufacturing company with a hierarchical structure and agents to represent each domain. In this case, both the Bureaucracy (section 2.3) and Supply Chain role models appear (see Fig 9). It is the responsibility of the Plant Manager to be the SC Successor to the Customer, but it is the Assembly (for example) functional group that requires input from Enterprise 2, so it is the SC Predecessor in that context. In Fig 9, the relevant role models appear in the top half of the diagram, while the entities in the application that play the roles appear in the bottom half. As in Fig 2, dashed lines indicate role assignments.

aPlantManager must play the whole role of Successor, in addition to the role of Manager. Likewise, anAssembly must be a Predecessor in addition to satisfying the responsibilities of a Subordinate. If negotiation and delivery are considered, the internal roles from section 3.3 appear as well. Both entities must appropriately address context switching as they go from role to role.

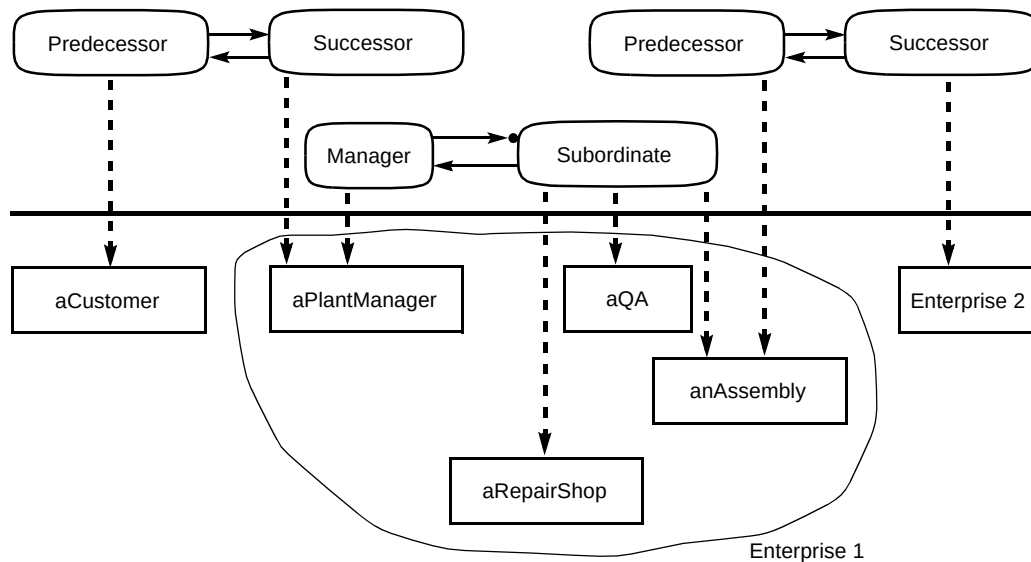


Fig 9 Bureaucracy with Supply Chain model for agent-enhanced workflow.

5. Agent design with role models

5.1 Role composition

Role models are also useful during agent system design. Basically, the roles that a given agent needs to play have to be identified and composed during design. This approach has been utilised in object-oriented software engineering, in particular during framework design [14]. An agent class can be designed by composing or assembling all of the responsibilities, interfaces, expertise, and protocols from the individual roles, following the documentation provided in Tables 1 and 2 in section 3, i.e each role must be fully specified according to the facets in Tables 1 and 2. If roles appear frequently enough, components can be built for the roles, and object-oriented implementations are discussed in section 5.2.

For example, if an agent can be found in a Supply Chain and also as a Manager, Subordinate, or Client (Director Client and Clerk Client are considered together for simplicity) of a Bureaucracy, the agent must be capable of playing the seven roles shown in Fig 10. Each of these roles would be fully specified, as per Tables 1 and 2.

5.2 Object-oriented designs of role models

Whenever possible, design patterns should be utilised to design and implement software systems. Several patterns [22, 23] exist for addressing applications where objects play roles; the role object pattern will be utilised here.

A design based on the role object pattern is depicted in Fig 11; it provides an object-oriented design (class diagram in UML notation) for an agent that participates in supply chains and bureaucracies and therefore must play all of the roles in Fig 10. (The design only addresses the role models;

additional classes would be required.) An agent is shown as comprised of a set of roles. A class is required for every role; the four roles that appear in the Supply Chain role sequence (Fig 7) are shown. As discussed in section 3.2, an SC Participant has added behaviour; its Customer, User, Provider, and Operator are refinements of the four base roles. This is depicted with an inheritance relationship in Fig 11. Further, the Manager, Subordinate, and Client (actually Clerk Client and Director Client) roles from the bureaucracy role model must be available. These are three additional role components, and Manager and Subordinate are subclasses of class Clerk.

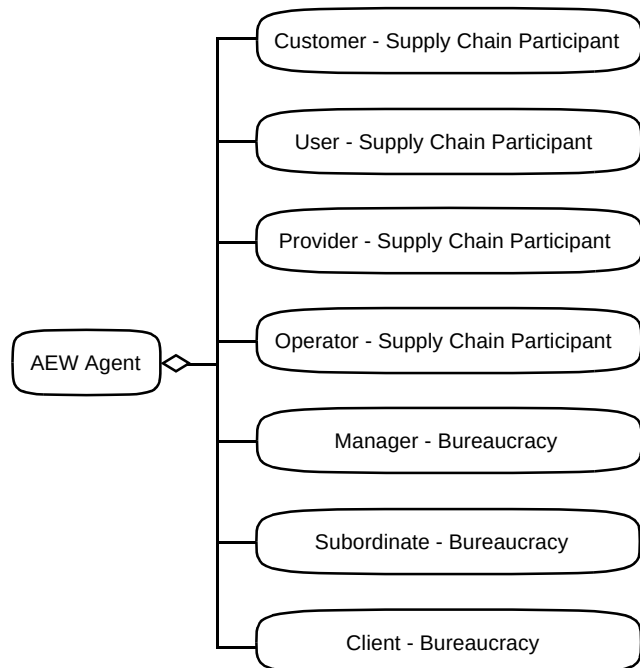


Fig 10 Role composition during design.

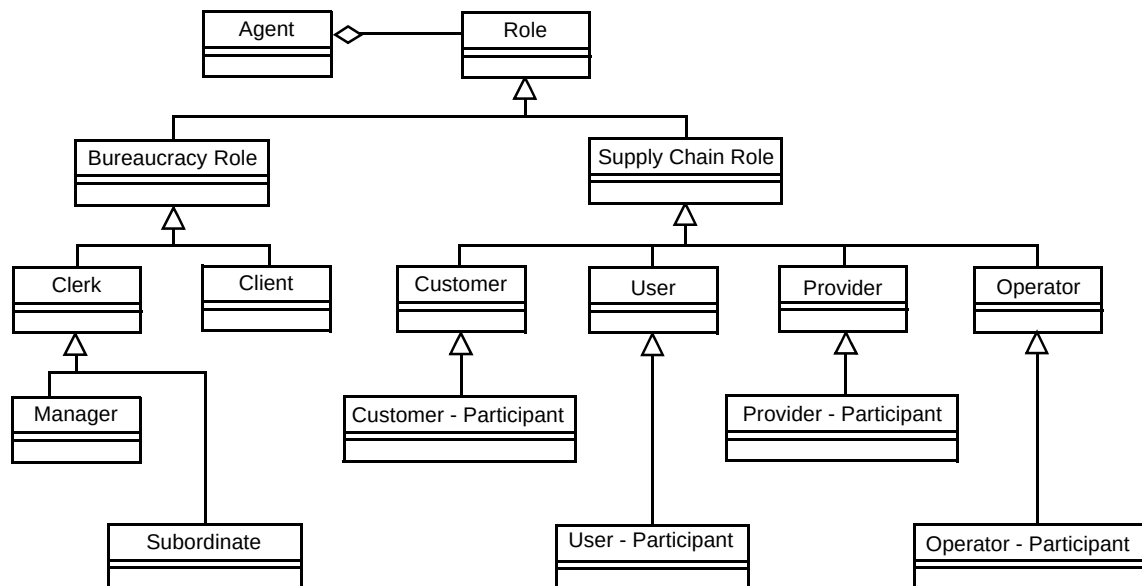


Fig 11 Role object pattern design for Supply Chain and Bureaucracy agent.

Agent implementations based on the role object pattern have been developed at BT and at RMIT. A hash-table or other container can be used to hold the roles, and all behaviour and context switching is delegated to them. A similar system, termed a plug-in architecture, has been developed to address mobile agents [24]. Role components are assembled to form an agent, as determined by the roles that must be played. Because code size is particularly important for mobile agents, only required roles are built into the agent.

6. Discussion and conclusions

6.1 Related concepts

Helm, Holland, and Gangopadhyay were the first to point out the need for techniques and support for specifying and abstracting inter-object behaviour [31]. A contract captures and abstracts behavioural dependencies between objects. It also defines a set of participants, along with their contractual obligations. Similar to role models, contracts are first class objects that can be generalised, refined and aggregated. The distinction between a contract and a role model is that a contract does not attempt to describe object interactions. It concentrates on specifying dependencies, constraints, and invariants.

Koubaris [27] and Ould [32] utilise roles in business process modelling, including the notions of role types and role instances. In Ould's models, a role is filled or played by an actor. As in object-oriented software engineering, business process roles can be transferred, shared, or distributed between entities (objects or actors).

Lupu and Sloman [33] emphasise managerial roles; the research is based on role theory [34], which describes a role

as a 'collection of rights and duties'. A role has a position within an organisation, and it has responsibilities made up of activities and required interactions. A role is within a domain (organisation), and the domain has policies that determine authorities and obligations for its member roles. Interestingly, this paper proposes a framework to provide support for communication and co-ordination in an environment where both human and automated agents coexist.

Mahling and Feirerstein [35] describe co-ordination as a set of concurrently co-operating roles, with interdependent actions and interactions. This research points out that CSCW should model and implement both automated and user-active role behaviour. Further, interactions should be formally controlled segments of behaviour, with state changes in the entities playing the roles.

Therefore, the concept of a role has appeared extensively in business process modelling, distributed systems, and CSCW. However, only role modelling, as defined in object-oriented software engineering, has formalised the concepts of a role and a role model sufficiently to enable their realisations in software.

6.2 Related research in agent analysis and design

The work presented here is similar and complementary to the MAS-Common KADS methodology that was developed by Iglesias et al [2, 3]. MAS-Common KADS extends common KADS with techniques from object-oriented software engineering and protocol engineering. While MAS-Common KADS employs the notion of an agent's role, it does not really define what this means. However, the tables in section 3.2 of this paper are our steps

towards defining agent roles. Roles are also used in the methodology for BDI agents developed by Kinny, Georgeff, and Rao [4]. In the BDI methodology, agent roles are identified, along with their associated responsibilities, services, and collaborators.

However, neither Iglesias et al [2, 3] nor Kinny et al [4] utilise role models, and the concepts of role and class are used interchangeably. In particular, Iglesias et al state that a CRC card describes an agent's class; in this paper, RRC (Role Responsibility Collaborators) cards document an agent's role. The distinction is important. As discussed in section 2, a class stipulates the capabilities of an individual object, while a role focuses on the position and responsibilities of an object in a structure (role model). Classes are templates for instantiating similar objects; role models are templates for instantiating subsystems of interacting, collaborating objects.

Another important distinction is that MAS-Common KADS and the BDI methodology are not pattern-based; they are general methodologies. In this paper, role models are introduced primarily as a way to document agent patterns of interaction. Role models can be employed on their own during analysis, before patterns have been identified; however, that is not the main aim of the research carried out at BT.

6.3 Related research in agent roles

Groups at Carnegie Melon University (CMU) [5] and the University of Toronto [1] have recently proposed role-based approaches to agent co-ordination. The University of Toronto approach includes a co-ordination language that defines agent organisations, agents, and roles. The agents play roles in an organisation, and a role is defined by its major function, permissions, obligations, and interdictions. The agent's beliefs and reasoning are partitioned on the basis of the roles it plays to facilitate context switching [1]. Agent behaviour is determined by conversation plans, and these are usually specified to be between a particular pair of roles.

CMU's [5] research on task decomposition, formations, and role assignments is a result of their work on robot soccer. CMU defines an agent role as a specification of an agent's internal and external behaviours. A formation is a team (or sub-team) structure that decomposes the task space into a set of roles. Formations are dynamic and flexible, and, in addition to specifying roles, they also specify inter-role interactions. The team's behaviour in a formation is governed by plans that involve multiple agents in various roles. Roles and formations are populated by actual agents or robots. There are initial formations (termed locker-room agreements), but there are also run-time triggers for dynamic changes.

The work presented in this paper emphasises the analysis and design phases of the agent software engineering life cycle. The work at CMU and the University of Toronto proposes sets of agent organisations and roles for applications in system administration, supply chain management, telecommunications, and robot soccer. However, the organisations and roles are identified on an *ad hoc* basis, with no justification other than that they mirror present structures found in real life. This may be totally appropriate for some applications, such as robot soccer, but other applications may require additional scrutiny.

Whereas some of the structures may be patterns, they need to be documented as such so that their context and motivation are recorded. Otherwise, they will continue to be employed in an *ad hoc*, and potentially inappropriate, manner.

This paper presents a methodology (pattern language) for identifying, specifying, designing, and implementing agent roles and role models. The role models defined by the groups at the University of Toronto and at CMU may well be incorporated into the pattern language.

6.4 Summary and future work

This paper extends object-oriented role modelling to provide a new approach to agent system engineering. In particular, it illustrates how role models can form the basis of a pattern language for analysing, designing, and implementing distributed, collaborative agent systems.

Role models are new abstractions for agent collaboration and mobility; they are also a new way to document patterns of interaction. Our experience last year in the intelligent business systems and intelligent systems groups at BT indicated that role models are useful for agent systems as concepts, analysis models, and templates for design. As discussed in sections 2.1 and 6.2, classes do not effectively capture collaboration and patterns of interaction. Collaboration is a crucial facet of agent behaviour; therefore, role models are a powerful abstraction for agent system development. It is anticipated that role models will be beneficial in downstreaming or transferring agent technology as they provide conceptual models for how agents can be used. In addition, role models lead to detailed designs and implementations for collaborative behaviour.

More complete definitions of agent roles and role models are required, as well as additional notations and formalisms. Agent role models also need to be documented in a pattern format so their context, applicability, and semantics are clear. The research is also being extended to cover alternative designs and implementations for role models.

Acknowledgements

Much of this research was carried out while the author was a senior research fellow in the Intelligent Business Systems group at BT Laboratories. The author also gratefully acknowledges the assistance of E Andersen.

References

- 1 Barbuceanu M, Gray T, Mankovski S: 'Co-ordinating with obligations', Agents '98, Minneapolis, pp 62—69 (May 1998).
- 2 Iglesias C A, Garijo M, Gonzalez J and Velasco J R: 'A methodological proposal for multiagent systems development extending common KADS', in Gaines B and Musen M (Eds): 'Proceedings of the 10th Banff Knowledge Acquisition for Knowledge-Based Systems Workshop', 1, (1996).
- 3 Iglesias C A, Garijo M, Gonzalez J C and Velasco J R: 'Analysis and design of multiagent systems using MAS-CommonKADS', in Fourth International Workshop on Agent Theories, Architectures and Languages ATAL'97 Rhode Island, USA, Springer (1997).
- 4 Kinny D, Georgeff M and Rao A: 'A methodology and modelling technique for systems of BDI', in MAAMAW'96, LNAI 1038, pp 56—71, The Netherlands (1996).
- 5 Veloso M, Stone P and Han K: 'The CMUnited-97 robotic soccer team: perception and multiagent control', Agents '98, Minneapolis, pp 78—86 (May 1998).
- 6 Andersen E: 'Conceptual modeling of objects: a role modeling approach', PhD Thesis, University of Oslo (1997).
- 7 Kristensen B C and May D C M: 'Activities: abstractions for collective behavior', Proceedings of the European Conference on Object-Oriented Programming (ECOOP'96), Linz, Austria (1996).
- 8 Kristensen B B and Olsson J: 'Roles and patterns in analysis, design and implementation', OOIS'96, Proceedings of the 3rd International Conference on Object-Oriented Information Systems, London, England (1996).
- 9 Kristensen B B and Osterbye K: 'Object-oriented modeling with roles', OOIS'95, Proceedings of the 2nd International Conference on Object-Oriented Information Systems, Dublin, Ireland (1995).
- 10 Reenskaug T, Wold P and Lehne O A: 'Working with objects, the OOram software engineering method', Manning Publications Co, Greenwich (1996).
- 11 Reenskaug T: 'Role modeling enters the main stream', Object EXPERT (January 1997).
- 12 Reenskaug T: 'The four faces of UML', (May 1998) — www.ifi.uio.no/~trygve/documents
- 13 Riehle D: 'Composite design patterns', OOPSLA '97, Proceedings of the 1997 Conference on Object-Oriented Programming Systems, Languages and Applications, ACM Press, pp 218—228 (1997).
- 14 Riehle D: 'Bureaucracy', in Martin R, Riehle D and Buschmann F (Eds): 'Pattern Languages of Program Design 3', Addison Wesley, pp 163—185 (1998).
- 15 Riehle D and Gross T: 'Role model based framework design and integration', OOPSLA'98, Proceedings of the 1998 Conference on Object-oriented Programming Systems, Languages and Applications, ACM Press (1998).
- 16 Rational Software, 'UML Documentation: Behavioral Elements Package: Collaboration Overview', — <http://www.rational.com/uml/resources/documentation/semantics/semant9a.jtmpl>
- 17 Gamma E R, Helm R, Johnson R and Vlissides J: 'Design patterns: elements of reusable object-oriented software', Addison-Wesley (1994).
- 18 Kendall E A, Malkoun M T and Jiang C H: 'The application of object-oriented analysis to agent based systems', The Report on Object Oriented Analysis and Design/The Journal of Object Oriented Programming (February 1997).
- 19 Kendall E A, Malkoun M T and Jiang C H: 'Multiagent system design based on object oriented patterns', The Report on Object Oriented Analysis and Design in conjunction with The Journal of Object Oriented Programming (June 1997).
- 20 Kendall E A, Murali Krishna P V, Pathak C V and Suresh C B: 'Patterns of intelligent and mobile agents', Agents '98 (May 1998).
- 21 Kendall E A, Palanivelan U and Kalikivayi S: 'Capturing and structuring goals: analysis patterns', EuroPlop '98, European Pattern Languages of Programming, Germany (July 1998).
- 22 Fowler M: 'Dealing with roles', Pattern Languages Of Programming (PLOP '97), Illinois (1997).
- 23 Baumer D, Riehle D, Siberski W and Wolf M: 'Role object', Pattern Languages Of Programming (PLOP) '97, Illinois (1997).
- 24 Tu M T, Griffel F, Merz M and Lamersdorf W: 'A plug-in architecture providing dynamic negotiation capabilities for mobile agents', Mobile Agents: Second International Workshop (MA'98), Stuttgart, Germany, pp 222—236 (September 1998).
- 25 Kendall E A: 'Agent roles and role models: new abstractions for multiagent system analysis and design', International Workshop on Intelligent Agents in Information and Process Management, German Conference on Artificial Intelligence, Bremen, Germany (September 1998).
- 26 Nwana H S, Ndumu D T and Lee L C: 'ZEUS: an advanced tool kit for engineering distributed multi-agent systems', Prac App Agent Multi-agent Sys (PAAM), pp 377—391 (March 1998).
- 27 Koubarakis M and Plexousakis D: 'Business process modelling and design, a formal model and methodology', BT Technol J, 17, No 4, pp 23—35 (October 1999).
- 28 Dickinson I: 'Agent standards', Agent Technology Group (1997) — <http://drogo.cse.stet.it/fipa>
- 29 Dunskus B V, Grecu D L, Brown D C and Berker I: 'Using single function agents to investigate conflicts', in Smith I (Ed): 'AI EDAM, Special Issue: Conflict Management in Design', Cambridge UP (1996).
- 30 Hickman F R, Killin J L, Land L, Mulhall T, Porter D and Taylor R M: 'Analysis for knowledge-based systems: a practical guide to the KADS methodology', Ellis Horwood, Chichester (1989).
- 31 Helm R, Holland I M and Gangopadhyay D: 'Contracts: specifying behavioral compositions in object-oriented systems', Object Oriented Programming, Systems and Languages, ECOOP/ OOPSLA '90 Proceedings, pp 169—180 (October 1990).
- 32 Ould M: 'Business processes: modelling and analysis for reengineering and improvement', John Wiley & Sons, West Sussex, England (1995).

- 33 Lupu E C and Sloman M: 'Towards a role-based framework for distributed systems management', *Journal of Network and Systems Management* (1996).
- 34 Biddle B J and Thomas E J: 'Role theory: concepts and research', R E Krieger Publishing Co (1979).
- 35 Mahling D and Feierstein M: 'Towards a model of cooperative action', *CSCW'91: Proceedings of the European Conference on Computer-Supported Co-operative Work* (1991).
- 36 Holt A, Ramsey W H R and Grimes J D: 'Coordination system technology as the basis for a programming environment', *Electrical Communication*, 57, pp 307—314 (1983).
- 37 Jacobson I et al: 'Object oriented software engineering: a use case driven approach', Addison Wesley (1992).



Professor Elizabeth A Kendall is the Sun Microsystems Chair of Network Computing at Monash University in Australia. She received her Bachelor of Science degree at the Massachusetts Institute of Technology and then completed an MSc and a PhD at the California Institute of Technology. From 1978 to 1990 she led and participated in research and development in advanced computing at major industrial laboratories in the USA, including Xerox, Boeing, and Lockheed. Since 1990, she has been an academic at universities in Australia and New Zealand, leading research in advanced object technology and agent systems. Her research in agent systems led to her being awarded a Senior Research Fellowship at the laboratories at BT Adastral Park.